

Ch8. Backpropagation

第 8 章反向传播

Section 8.1. Evaluation of Gradients

第 8.1 节梯度计算

8.1.3. A simple example

8.1.3. 简单示例

8.1.4. Numerical differentiation

8.1.4. 数值微分法

《深度学习基础》课程小组

2026 年 8 月 4 日

1 P1 8.1.3 A simple example

The above derivation of the backpropagation procedure allowed for general forms for the error function, the activation functions, and the network topology.

To illustrate the application of this algorithm, we consider a two-layer network of the form illustrated in Figure 6.9, together with a sum-of-squares error.

The output units have linear activation functions, so that $y_k = a_k$, and the hidden units have sigmoidal activation functions given by

$$h(a) \equiv \tanh(a) \tag{8.15}$$

where $\tanh(a)$ is defined by (6.14).

A useful feature of this function is that its derivative can be expressed in a particularly simple form:

$$h'(a) = 1 - h(a)^2. \tag{8.16}$$

We also consider a sum-of-squares error function, so that for data point n the error is given by

$$E_n = \frac{1}{2} \sum_{k=1}^K (y_k - t_k)^2 \tag{8.17}$$

where y_k is the activation of output unit k , and t_k is the corresponding target value for a particular input vector $\mathbf{x}_n$.

翻译：上述反向传播过程的推导允许误差函数、激活函数以及网络拓扑采用一般形式。

为了说明该算法的应用，我们考虑如图 6.9 所示形式的两层网络，并结合平方和误差函数。

输出单元具有线性激活函数，即 $y_k = a_k$ ，而隐藏单元具有 S 型激活函数，其定义为：

$$h(a) \equiv \tanh(a) \quad (8.15)$$

其中 $\tanh(a)$ 由公式 (6.14) 定义。

该函数的一个有用特性是，其导数可以用一种特别简单的形式来表示：

$$h'(a) = 1 - h(a)^2. \quad (8.16)$$

我们还考虑平方和误差函数，因此对于数据点 n ，误差由下式给出：

$$E_n = \frac{1}{2} \sum_{k=1}^K (y_k - t_k)^2 \quad (8.17)$$

其中 y_k 是输出单元 k 的激活值，而 t_k 是对应于特定输入向量 $\mathbf{x}_n$ 的目标值。

2 P2

For each data point in the training set in turn, we first perform a forward propagation using

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i \quad (8.18)$$

$$z_j = \tanh(a_j) \quad (8.19)$$

$$y_k = \sum_{j=0}^M w_{kj}^{(2)} z_j \quad (8.20)$$

where D is the dimensionality of the input vector $\mathbf{x}$ and M is the total number of hidden units.

Also we have used $x_0 = z_0 = 1$ to allow bias parameters to be included in the weights.

Next we compute the δ 's for each output unit using

$$\delta_k = y_k - t_k. \quad (8.21)$$

Then, we backpropagate these errors to obtain δ 's for the hidden units using

$$\delta_j = (1 - z_j^2) \sum_{k=1}^K w_{kj}^{(2)} \delta_k, \quad (8.22)$$

which follows from (8.13) and (8.16).

Finally, the derivatives with respect to the first-layer and second-layer weights are given by

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i, \quad \frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j. \quad (8.23)$$

翻译:

对于训练集中的每个数据点，我们首先使用以下公式执行前向传播：

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i \quad (8.18)$$

$$z_j = \tanh(a_j) \quad (8.19)$$

$$y_k = \sum_{j=0}^M w_{kj}^{(2)} z_j \quad (8.20)$$

其中 D 是输入向量 $\mathbf{x}$ 的维度， M 是隐藏单元的总数。

此外，我们设定 $x_0 = z_0 = 1$ ，以便将偏置参数包含在权重中。

接下来，我们使用以下公式计算每个输出单元的 δ 值：

$$\delta_k = y_k - t_k. \quad (8.21)$$

然后，我们将这些误差反向传播，以使用以下公式获取隐藏单元的 δ 值：

$$\delta_j = (1 - z_j^2) \sum_{k=1}^K w_{kj}^{(2)} \delta_k, \quad (8.22)$$

该式由公式 (8.13) 和 (8.16) 推导得出。

最后，关于第一层和第二层权重的导数由下式给出：

$$\frac{\partial E_n}{\partial w_{ji}^{(1)}} = \delta_j x_i, \quad \frac{\partial E_n}{\partial w_{kj}^{(2)}} = \delta_k z_j. \quad (8.23)$$

注： δ_k 是输出层的误差， δ_j 是隐藏层的误差。

3 P3 8.1.4 Numerical differentiation

One of the most important aspects of backpropagation is its computational efficiency.

To understand this, let us examine how the number of compute operations required to evaluate the derivatives of the error function scales with the total number W of weights and biases in the network.

A single evaluation of the error function (for a given input data point) would require $\mathcal{O}(W)$ operations, for sufficiently large W .

This follows because, except for a network with very sparse connections, the number of weights is typically much greater than the number of units, and so the bulk of the computational effort in forward propagation arises from evaluation of the sums in (8.5), with the evaluation of the activation functions representing a small overhead.

Each term in the sum in (8.5) requires one multiplication and one addition, leading to an overall computational cost that is $\mathcal{O}(W)$.

翻译：

反向传播最重要的特性之一是其计算效率。

为了理解这一点，让我们考察一下评估误差函数导数所需的计算操作数量是如何随网络中权重和偏置的总数 W 缩放的。

对于足够大的 W ，单次评估误差函数（对于给定的输入数据点）将需要 $\mathcal{O}(W)$ 次操作。

这是因为，除了连接非常稀疏的网络外，权重的数量通常远大于单元的数量，因此前向传播中的大部分

计算量来自于评估公式 (8.5) 中的求和项，而激活函数的评估仅代表很小的额外开销。

公式 (8.5) 中求和项的每一项都需要一次乘法和一次加法，这使得整体的计算成本为 $\mathcal{O}(W)$ 。

4 P4

An alternative approach to backpropagation for computing the derivatives of the error function is to use finite differences.

This can be done by perturbing each weight in turn and approximating the derivatives by using the expression

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \epsilon) - E_n(w_{ji})}{\epsilon} + \mathcal{O}(\epsilon) \quad (8.24)$$

where $\epsilon \ll 1$.

In a software simulation, the accuracy of the approximation to the derivatives can be improved by making ϵ smaller, until numerical round-off problems arise.

The accuracy of the finite differences method can be improved significantly by using symmetrical central differences of the form

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \epsilon) - E_n(w_{ji} - \epsilon)}{2\epsilon} + \mathcal{O}(\epsilon^2). \quad (8.25)$$

In this case, the $\mathcal{O}(\epsilon)$ corrections cancel, as can be verified by a Taylor expansion of the right-hand side of (8.25), and so the residual corrections are $\mathcal{O}(\epsilon^2)$.

Note, however, that the number of computational steps is roughly doubled compared with (8.24).

Figure 8.2 shows a plot of the error between a numerical evaluation of a gradient using both finite differences (8.24) and central differences (8.25) versus the analytical result, as a function of the value of the step size ϵ .

翻译：

作为反向传播的替代方案，计算误差函数导数的一种方法是使用有限差分法。

这可以通过依次扰动每个权重，并使用以下表达式来近似导数来实现：

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \epsilon) - E_n(w_{ji})}{\epsilon} + \mathcal{O}(\epsilon) \quad (8.24)$$

其中 $\epsilon \ll 1$ 。

在软件模拟中，可以通过减小 ϵ 的值来提高导数近似的精度，直到出现数值舍入问题为止。

通过使用如下形式的对称中心差分法，可以显著提高有限差分法的精度：

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{E_n(w_{ji} + \epsilon) - E_n(w_{ji} - \epsilon)}{2\epsilon} + \mathcal{O}(\epsilon^2). \quad (8.25)$$

在这种情况下， $\mathcal{O}(\epsilon)$ 的修正项会相互抵消，这可以通过对公式 (8.25) 右侧进行泰勒展开来验证，因此剩余的修正项为 $\mathcal{O}(\epsilon^2)$ 。

然而需要注意的是，与公式 (8.24) 相比，计算步骤的数量大约增加了一倍。

图 8.2 展示了使用有限差分法 (8.24) 和中心差分法 (8.25) 进行梯度数值评估时，其与解析结果之间的误差随步长 ϵ 取值变化的曲线图。

5 P5

The main problem with numerical differentiation is that the highly desirable $\mathcal{O}(W)$ scaling has been lost.

Each forward propagation requires $\mathcal{O}(W)$ steps, and there are W weights in the network each of which must be perturbed individually, so that the overall computational cost is $\mathcal{O}(W^2)$.

However, numerical differentiation can play a useful role in practice, because a comparison of the derivatives calculated from a direct implementation of backpropagation, or from automatic differentiation, with those obtained using central differences provides a powerful check on the correctness of the software.

翻译：

数值微分的主要问题是失去了极其理想的 $\mathcal{O}(W)$ 缩放特性。

每次前向传播都需要 $\mathcal{O}(W)$ 步操作，而网络中共有 W 个权重，每一个都必须被单独扰动，因此总体的计算成本为 $\mathcal{O}(W^2)$ 。

然而，数值微分在实践中仍然可以发挥有用的作用，因为将直接实现反向传播或自动微分计算出的导数与使用中心差分法得到的导数进行比较，是检验软件正确性的有力手段。